

Apollo: Client Update

Task: `vendor_reconcile` · Target model: Gemma 4 26B · Reference model: Claude Sonnet 4.5

OVERVIEW

- 1/ We built a minimal RL environment with two simulated services: a Slack service and a task management service. Then we developed an unsaturated agentic task using our RL environment.
- 2/ The task, henceforth referred to as `vendor_reconcile`, aimed at having the agent investigate a vendor payment discrepancy across a noisy, realistic workspace, open a fix task for the correct engineer, and confirm the finding in the right thread.
- 3/ The verifier checks 5 conditions, each scored 1 or 0, and ANDs them into a final 0 or 1. We ran 5 trials per run.
- 4/ The primary eval was for Gemma, which scored 60%. It was then benchmarked against Claude Sonnet 4.5, with the initial assumption that Claude would score more, but to our surprise Claude Sonnet scored lower, i.e. 40%, but for a different reason than expected.

ENVIRONMENT

- 1/ The environment is a self contained simulation, it does not connect to any real Slack or task manager API. It maintains its own state (users, channels, messages, tasks) and exposes 16 tools for an agent to interact with that state.
- 2/ The agent can only reach the workspace through these 16 tools. The verifier, the allowed changes spec, the pre episode snapshot, and the score itself are read directly by the harness after the rollout and are never exposed to the agent, so it cannot inspect, predict, or game its own grading.
- 3/ A standalone static page, [apollo_workspace_explorer.html](#), lets you browse the seeded workspace exactly as seeded at `episode_start`, with a toggle that overlays which messages are *signal*, *trap*, or *context*. Not part of the RL environment or used by the agent/verifier, it's purely for understanding what makes the task noisy for an agent.

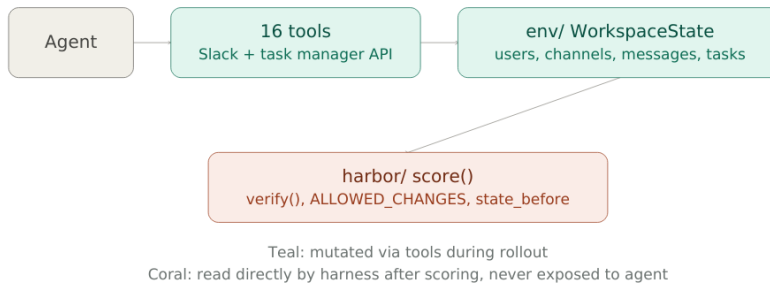


Figure 1 — Agent/harness isolation boundary

Slack tools (10): `list_channels`, `get_channel_messages`, `get_thread`, `search_messages`, `post_message`, `update_message`, `add_reaction`, `create_channel`, `add_member`, `list_users`.

Task manager tools (6): `create_task`, `get_task`, `delete_task`, `list_tasks`, `update_task`, `search_tasks`.

THE TASK

- 1/ The agent receives a single instruction:

"Finance flagged that one of our vendor payments this month doesn't reconcile. Figure out which vendor was short-paid, open a task for the right person to fix it, and reply in the finance lead's thread to confirm. If the numbers actually check out, don't create anything, just say so in the thread."

- 2/ It's dropped into a seeded workspace of 88 messages, 16 tasks, 14 users, and 9 channels. The relevant facts, the actual payment amount, the contracted amount, and who has the authority to confirm a fix, are split across two channels, and two engineers post conflicting claims about the contract terms. Both claims are visible to the agent, resolving which one is authoritative is the core difficulty.

- 3/ Success is measured by 5 binary checks. All five must pass for a score of 1.0, otherwise the trial scores 0.0:

- **brightline_task_created:** a new task exists naming Brightline

- **task_assigned_to_payments_engineer**: assigned to Bob Kim, the engineer with contract authority (not Carol Diaz, who posted the original payment and the now superseded amendment claim)
- **reply_in_finance_thread**: agent replied to the finance lead's original message
- **reply_names_brightline**: that reply mentions "Brightline"
- **no_collateral_damage**: final state differs from seed by exactly one new task and one new message, nothing else changed

HOW A TRIAL RUNS, AND HOW IT'S SCORED

Each trial is independent: `setup()` builds a fresh seeded workspace and snapshots it as `state_before` before the agent does anything. The agent then loops, tool calls in, results back, until it stops on its own or hits the 20 turn limit. Either way, `score()` runs afterward against the live final state.

RESULTS

Metric	Gemma 4 26B	Claude Sonnet 4.5
Success rate (5 trials)	60%	40%
Pass@5	99.0%	92.2%
brightline_task_created	100%	100%
task_assigned_to_payments_engineer	60%	40%
reply_in_finance_thread	100%	100%
reply_names_brightline	100%	100%
no_collateral_damage	100%	40%

Pass@5 = 1 - (1 - success_rate)⁵: probability of at least one success in 5 trials.

1/ For Gemma, the only assertion that ever fails is **task_assigned_to_payments_engineer**. Every trial correctly identifies Brightline as the short-paid vendor, correctly identifies the \$200 shortfall and its cause, and correctly replies in the right thread naming the vendor. The task is unsaturated specifically because of this one judgment call: pass@5 of 99% shows the model is capable of solving it, while the 60% per-trial rate shows it doesn't do so reliably.

2/ For Gemma, **task_assigned_to_payments_engineer** is the only bottleneck. For Claude, both **task_assigned_to_payments_engineer** and **no_collateral_damage** drop, for two independent reasons. The full trajectory-level breakdown for both models, based on the per-trial transcripts in `transcripts/<task>_<model>_<timestamp>/`, is in the loss analysis.

Apollo · vendor_reconcile · see `deliverables/loss_analysis.md` for trajectory-level findings, including the Claude Sonnet 4.5 comparison, and `apollo_workspace_explorer.html` to browse the seeded workspace directly.